Source: docs\SHOPPING.md | Export: 11 November 2026

Account-linked Bag and Wishlist

Public browsing remains anonymous. Shopping actions require the existing Firebase email/password session; email verification and OTP are not added. The current Bag/Wishlist card layouts, brand colours, header and five-tab navigation are reused. Checkout, orders, coupons, shipping and inventory reservations remain unavailable.

Persistence and ownership

- `customerShopping/{uid}/bag/{productId}--simple` or `{productId}--v-{variantId}`: `productId`, `variantId` (null for simple stock), positive integer `quantity`, server `createdAt`/`updatedAt`.
- `customerShopping/{uid}/wishlist/{productId}`: `productId`, server `createdAt`/`updatedAt`. One entry per product, including published products with zero stock.
- `customerShopping/{uid}/operations/{operationId}`: server-only hash, outcome and server creation timestamp. Receipts retain idempotency across uncertain retries and remove/re-add races. They contain no product descriptions, credentials or payment information and are retained; no TTL policy is required for this release.

Rules permit customers to read only their own bag/wishlist subcollections. All browser writes and all operation-receipt reads are denied. The existing `/api/catalog` handles authenticated `shopping:get`, `bag:add`, `bag:adjust`, `bag:remove` and `wishlist:set` actions. It verifies the Firebase token with revocation checking and project audience, derives UID paths exclusively from that token and rejects ownership/price/stock/unknown request fields. Product administration and login appearance retain both `admin: true` and enabled `adminAccess/{uid}` checks.

The API validates product/variant references and quantities, checks current publication/category/variant stock in transactions, and uses operation receipts to prevent repeated mutations. Quantity adjustments are atomic deltas, rather than overwriting a stale client quantity. Repeated Add never increases an existing line. Reducing an already excessive quantity is allowed so customers can recover after stock decreases. Removing unavailable lines remains possible.

Each list is capped at 100 items. List resolution batches only those product IDs and their distinct categories; it does not scan the catalog. It returns public catalog projections only. Draft/archived/deleted/disabled-category references retain removable lines with generic unavailable copy and no restricted details. Subtotals use integer paise, current selling prices and exclude unavailable or excessive-quantity lines. Stock is not reserved or deducted.

Browser behavior

Only the active UID has two bounded Firestore reference subscriptions. Account changes immediately mask previous account data; listeners detach and stale resolutions are ignored. Changes on another device trigger fresh API catalog resolution. Current price and availability are rechecked on shopping mutations, bag/wishlist navigation and refresh. There is no continuous subscription to every product or a focus-triggered loading overlay.

Signed-out actions keep a short-lived, tab-scoped **intent only** in sessionStorage (action, product/variant identifiers, quantity, random operation ID and timestamp). This is not a guest bag/wishlist, and legacy preview data is never imported. Existing safe internal return validation is reused. After sign-in/registration, the action is bound to that UID, current product state is checked, and the intent is consumed. A missing colour routes to product selection. Leaving the auth flow while signed out, invalid/expired intents and account mismatch clear it. Requests retried after uncertain failures reuse the operation ID; reload/back cannot repeat a consumed intent.

No optimistic persisted changes are shown: controls lock while requests are pending, successful mutations refresh authoritative state, failures retain saved state with a retry action. Loading, empty and error states are distinct. Navigation badges use saved bag quantity and unique wishlist count; zero badges are hidden. Image failures preserve the image frame and render an accessible fallback.

Release and owner acceptance

Publish the narrow `customerShopping` read-only owner rule block while preserving the deployed baseline. No composite indexes, new environment variables, Cloudinary changes or additional serverless functions are required. Runtime packaging must include the new `server/shopping.ts` import through the existing catalog function.

Automated shopping mutations use guarded loopback emulator accounts and fixtures only. After release, the owner should sign in with an authorized account and check:

1. Signed-out Add / heart -> login or registration -> action once; repeat Add keeps quantity unchanged.
2. Choose a colour, save it, change quantity in Bag and verify counts. Wishlist Add keeps the saved heart.
3. Refresh and sign in on another device: saved items and changes synchronize.
4. Sign out and use another account: no previous items or counts appear.

5. Remove an unavailable saved item; checkout remains disabled.

Local/emulator success does not establish authenticated production behavior. Do not use the owner's real shopping records for automated mutation tests.